

Réparer une application React

Candidat : Sarah Brahimi · À rendre le dimanche 30 août 2026 · Patrick Kusanagi

Durée estimée : 2 à 3 heures. Stack : React + Vite. Le projet de départ t'est fourni séparément.

Contexte

Pendant ton stage, tu ne partiras presque jamais d'une page blanche. Tu reprendras du code écrit par quelqu'un d'autre, souvent bancal, et tu devras le faire fonctionner sans tout casser. C'est le cœur du travail de maintenance que je fais pour mes clients.

Cet exercice reproduit exactement cette situation : je te donne une petite application React de gestion de tâches. **Elle fonctionne mal : elle contient cinq bugs.**

Ce que je regarde

Ce n'est **pas** « est-ce que ça marche à la fin ». C'est **comment tu raisones pour y arriver**. Trouver un bug par hasard et le trouver parce qu'on a compris le problème, ce n'est pas la même chose — et ça se voit immédiatement dans l'explication que tu en donnes.

Le travail demandé

- 1 Lance le projet et vérifie que tu le vois tourner dans ton navigateur.
- 2 Utilise l'application comme le ferait un utilisateur, et **repère les cinq comportements anormaux**. Pense à ouvrir la console du navigateur : elle parle.
- 3 Corrige-les, avec **un commit par bug**.
- 4 Rédige un fichier `FIX.md` à la racine (format ci-dessous).

```
npm install
npm run dev
```

Le fichier FIX.md

C'est la partie la plus importante de l'exercice. Pour **chacun** des cinq bugs, trois lignes :

- **Symptôme** — ce que l'utilisateur voit, décrit sans jargon.
- **Cause** — pourquoi ça arrive, techniquement.
- **Correction** — ce que tu as changé, et pourquoi cette solution plutôt qu'une autre.

Exemple du niveau de détail attendu :

```
## Bug 3 – La liste se vide quand on coche une case

**Symptôme** : quand je coche une tâche, toutes les autres disparaissent
de l'écran jusqu'au rechargement de la page.

**Cause** : le state est remplacé au lieu d'être mis à jour – setTasks reçoit
uniquement la tâche modifiée au lieu du tableau complet.

**Correction** : je reconstruis un nouveau tableau avec .map() en ne
remplaçant que l'élément concerné, pour ne pas muter le state existant.
```

Bonus, si tu as le temps

Ajoute un filtre permettant d'afficher *toutes les tâches*, *seulement celles en cours* ou *seulement celles terminées*. À faire uniquement une fois les cinq bugs corrigés et documentés — le bonus ne rattrape pas un FIX.md bâclé.

Ce qui n'est pas demandé

Pas de refonte graphique, pas de bibliothèque supplémentaire, pas de tests automatisés, pas de déploiement. Ne réécris pas l'application : répare-la. Si tu es tentée de tout refaire, c'est justement le réflexe que cet exercice cherche à travailler.

Consignes communes aux deux exercices

Elles comptent dans l'évaluation autant que le code lui-même.

- Un dépôt **GitHub public** (un par exercice, ou un dépôt avec deux dossiers).
- Tu travailles **sur une branche**, jamais directement sur `main`. Une branche par exercice.
- **Commits atomiques** : un commit = un changement qui se raconte en une phrase. Évite les messages « fix », « update » ou « modif ».
- Quand tu as fini, tu ouvres une **Pull Request** de ta branche vers ton `main`, et tu décris dedans ce que tu as fait et pourquoi. **Tu ne la merges pas** : je la relirai comme je relis celles de mes clients.
- Tu n'as jamais ouvert de Pull Request : c'est volontaire. La documentation GitHub est claire, débrouille-toi, et écris-moi si tu bloques — me demander de l'aide n'est pas un mauvais point.
- Un fichier `README.md` à la racine : comment lancer le projet, ce que tu as fait, ce que tu n'as pas réussi, et le temps que tu y as passé.
- Tu peux utiliser l'IA. Mais tu dois pouvoir m'expliquer **chaque ligne** que tu rends : je poserai des questions précises dessus.

Si tu bloques

Si un point te bloque plus d'une heure, **note-le dans le README et passe à la suite**. Un exercice à moitié fini mais bien expliqué vaut mieux qu'un exercice fini que tu ne sais pas justifier. Et tu peux m'envoyer un message à tout moment : savoir demander de l'aide au bon moment fait partie du métier.